Das Gerüst

Termin 2 · 06.10.2026

FÄLLIG
bis Termin 3 am 13.10.

ZEITBEDARF
ca. 2 Stunden

BAUSTEINE IM REPO
t2-semantisches-geruest, t2-formular, t2-barrierefreiheit

KONTAKT
dhbw@theaisoftwarecompany.com

Euer Projekt bekommt heute Knochen: Seiten, Navigation, ein Formular. Alles noch ungestaltet – und genau das ist der Punkt. Wenn die Struktur ohne CSS lesbar ist, stimmt sie.

Teil A · Pflicht

A1 Seiten und Navigation

Baut **zwei bis drei Seiten** (z. B. Start, Übersicht, Über) oder eine Single-Page mit Ankern. Jede Seite hat:

- das vollständige Grundgerüst mit `lang`, `charset`, `viewport`, `title`
- `header` mit `nav`, ein `main`, ein `footer`
- genau **eine** `h1`, darunter `h2` / `h3` in der richtigen Reihenfolge

Keine `div`-Suppe. Wenn ein Element eine Bedeutung hat, gibt es dafür ein Element.

→ **Fertig, wenn** man von jeder Seite zu jeder anderen kommt und die Gliederung in den DevTools ohne Rätselraten lesbar ist.

A2 Ein Formular, das sich selbst prüft

Mindestens drei Felder mit **unterschiedlichen** `type`-Werten, dazu ein `button type="submit"`.

- jedes Feld hat ein `label` mit `for` auf die `id`
- jedes Feld hat ein sinnvolles `name`
- mindestens ein Feld ist `required` und hat eine zweite Einschränkung (`minlength`, `min` / `max`, `pattern`)

Kein JavaScript. Der Browser kann das.

→ **Fertig, wenn** das Absenden mit leerem Pflichtfeld scheitert, mit gültigen Werten klappt – und ein Klick auf den Text „E-Mail" den Cursor ins Feld setzt.

A3 Barrierefreiheits-Durchgang

1. Geht **einmal komplett ohne Maus** durch eure Seite: nur `Tab`, `Shift+Tab`, `Enter`.
2. DevTools → Lighthouse → nur „Accessibility" → **Analyze**.
3. Behebt, was die Liste anmeckert.

→ **Fertig, wenn** jede Aktion per Tastatur erreichbar ist und Lighthouse **mindestens 90** Punkte für Accessibility zeigt.

A4 Der `head` und die Maschinen

Ergänzt auf jeder Seite `title` und `meta name="description"`. Legt außerdem im Wurzelverzeichnis an:

- `robots.txt` mit `User-agent: *` und `Allow: /`
- `sitemap.xml` mit euren Seiten

→ **Fertig, wenn** beide Dateien unter ihren URLs erreichbar sind und jede Seite einen eigenen, aussagekräftigen `title` hat.

A5 Git ernst nehmen

Mindestens **drei Commits** über die Woche verteilt, mit Nachrichten, die sagen, was passiert ist. Im Zweierteam committet jede Person selbst.

→ **Fertig, wenn** `git log --oneline` eine Historie zeigt, die man lesen kann. „update" dreimal untereinander ist keine.

Teil B · Bonus (+1 Punkt)

Macht eure Seite maschinenlesbar: ein `llms.txt` im Wurzelverzeichnis **und** ein JSON-LD-Block nach schema.org im `head` der Startseite.

Prüft das JSON-LD mit dem Rich Results Test von Google. Ein Screenshot des Ergebnisses kommt in die README.

Teil C · Verständnisfragen

1. Warum ist `<button>` besser als `<div onclick="...">`? Nennt zwei Gründe, die nichts mit Aussehen zu tun haben.
2. Wofür ist das `name`-Attribut in einem Formular da – und was passiert, wenn es fehlt?
3. Ihr habt zwei `h1` auf einer Seite. Wen stört das, und warum?

So arbeitest du hier mit KI

Heute kommt Stufe 2 dazu: Copilot im Editor, als Erklärer.

Der nützlichste Handgriff dieser Woche: Markiert euer HTML und fragt

> *„Ist dieses HTML semantisch korrekt? Nenne mir die Stellen, an denen ein spezifischeres Element passen würde – aber schreib den Code nicht um, ich will es selbst ändern."*

Und nach dem Lighthouse-Lauf:

> *„Lighthouse meldet: . Erklär mir, was dahintersteckt und warum es für einen Screenreader ein Problem ist."*

Achtung, typische Falle: Auf die Bitte „mach mir eine Seite" liefern Modelle fast immer `div`-Suppe mit CSS-Klassen. Das ist kein Zufall, sondern Durchschnitt aus Trainingsdaten. Ihr sollt es besser machen.
